

Multilayer Perceptron: Manual Implementation and Training on MNIST

Ahmad Mustafa¹

National University of Computer and Emerging Sciences (FAST-NUCES),
Islamabad, Pakistan
`i252565@isb.nu.edu.pk`

Abstract. This report presents a complete manual implementation of a Multilayer Perceptron (MLP) neural network. Starting from first principles, we implement forward propagation, loss calculation using Mean Squared Error, backpropagation using the chain rule, and gradient descent weight updates. We first apply these steps manually on a small student performance dataset with three samples and two features. The network architecture consists of one input layer, two hidden layers with sigmoid activation, and one output layer. We then extend the implementation to Python using NumPy and train on the MNIST handwritten digit dataset achieving a test accuracy of 95.54%. All computations including forward pass, backpropagation, and weight updates are implemented from scratch without any deep learning frameworks.

Keywords: Multilayer Perceptron · Backpropagation · Gradient Descent · MNIST · Neural Networks

1 Introduction

Artificial neural networks are the foundation of modern deep learning. A Multilayer Perceptron (MLP) is the simplest form of a deep neural network, consisting of an input layer, one or more hidden layers, and an output layer. Each neuron computes a weighted sum of its inputs and passes the result through a nonlinear activation function. Training an MLP requires two phases: a forward pass to compute predictions, and a backward pass to compute gradients and update weights using gradient descent.

In this project we manually implement every step of MLP training on a small binary classification dataset, then scale the implementation to the MNIST dataset using NumPy. This exercise builds deep intuition for how neural networks learn.

2 Background

2.1 MLP Architecture

An MLP consists of layers of neurons. Each neuron j in layer l computes:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b^{(l)} \quad (1)$$

$$a_j^{(l)} = \sigma(z_j^{(l)}) \quad (2)$$

2.2 Sigmoid Activation

The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (3)$$

Its derivative is:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) = a(1 - a) \quad (4)$$

2.3 Loss Function

We use Mean Squared Error (MSE):

$$L_{MSE} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2 \quad (5)$$

2.4 Gradient Descent Variants

Three variants of gradient descent were studied:

- **Batch GD:** Uses all samples per update. Stable but slow.
- **SGD:** Uses one sample per update. Fast but noisy.
- **Mini-Batch GD:** Uses a batch of samples. Best of both worlds and industry standard.

2.5 Optimizers

Momentum accumulates velocity to accelerate learning:

$$v_t = \beta v_{t-1} + (1 - \beta) \frac{\partial L}{\partial w}, \quad w \leftarrow w - \eta v_t \quad (6)$$

NAG improves momentum by computing gradient at lookahead position:

$$\tilde{w} = w - \eta \beta v_{t-1}, \quad g_t = \frac{\partial L(\tilde{w})}{\partial \tilde{w}} \quad (7)$$

3 Implementation

3.1 Network Architecture

The manual network has the following structure:

- Layer 0: Input layer — 2 neurons
- Layer 1: Hidden layer 1 — 2 neurons, Sigmoid
- Layer 2: Hidden layer 2 — 2 neurons, Sigmoid
- Layer 3: Output layer — 1 neuron, Sigmoid

Weights assigned for Roll No 25I-2565: $w_1 = -0.15, w_2 = -0.25, w_3 = -0.32, w_4 = 0.86, w_5 = 0.14, w_6 = 0.05, w_7 = -0.66, w_8 = 0.72, w_9 = -0.84, w_{10} = 0.53, b^{(1)} = -0.42, b^{(2)} = -0.04$.

3.2 Forward Pass

For sample $s^{(1)}$ with $x_1 = 0.2, x_2 = 0.8$:

- $z_1^{(1)} = -0.7060, a_1^{(1)} = 0.3305$
- $z_2^{(1)} = 0.2180, a_2^{(1)} = 0.5543$
- $z_1^{(2)} = -0.3596, a_1^{(2)} = 0.4111$
- $z_2^{(2)} = 0.3756, a_2^{(2)} = 0.5928$
- $z^{(3)} = -0.0311, \hat{y} = 0.4922$

3.3 Backpropagation

Using the chain rule, gradients were computed for all weights. Key values:

- $\delta^{(3)} = -0.2538$
- $\frac{\partial L}{\partial w_9} = -0.1043, \frac{\partial L}{\partial w_{10}} = -0.1505$
- $\delta_1^{(2)} = 0.0516, \delta_2^{(2)} = -0.0325$

3.4 Weight Update

Using learning rate $\eta = 0.1$, weights were updated using:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w} \quad (8)$$

After one update: $w_9^{new} = -0.8296, w_{10}^{new} = 0.5451$.

4 Experiments

4.1 Manual Dataset Results

The MSE loss after 5 iterations decreased from 0.2528 to 0.2470, confirming the network is learning correctly with $\eta = 0.1$.

4.2 MNIST Results

The Python implementation was trained on MNIST with the following configuration:

- Architecture: $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$
- Learning rate: 0.1
- Epochs: 20
- Batch size: 32

The training loss decreased from 0.0296 to 0.0069 over 20 epochs. Final test accuracy achieved was **95.54%**.

5 Discussion

The results confirm that a simple MLP trained with mini-batch gradient descent can achieve strong performance on MNIST. The loss decreased consistently across all epochs demonstrating effective learning. One sample (digit 5) was misclassified as 6, which is expected given the visual similarity between these digits. Using ReLU activation instead of sigmoid would likely improve performance further by addressing the vanishing gradient problem.

6 Conclusion

This project successfully implemented a Multilayer Perceptron from scratch, covering forward propagation, MSE loss, backpropagation, gradient descent, mini-batch training, and momentum-based optimizers. The manual calculations provided deep understanding of every computation inside a neural network. The Python implementation on MNIST achieved 95.54% test accuracy, demonstrating the effectiveness of the approach.

References

1. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Nature* 323, 533–536 (1986)
2. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86(11), 2278–2324 (1998)